

TRABALHO 2 DE PROGRAMAÇÃO DE SOFTWARE BASICO

Feito por: Henrique Knack

1. INTRODUÇÃO

Este trabalho consiste no desenvolvimento de uma biblioteca de gerenciamento de memória em C. A biblioteca implementa um alocador baseado em pool: um bloco contíguo de memória é reservado na inicialização e todas as alocações subsequentes são servidas a partir desse pool, sem chamadas adicionais ao sistema operacional. São suportadas três estratégias de alocação: First Fit, Best Fit e Worst Fit.

1.1 CODIGO FONTE

Implementação de todas as funções:

`mymemory_init`: o pool de memória e define a estratégia de alocação

`mymemory_alloc`: aloca um bloco de tamanho solicitado dentro do pool

`mymemory_free`: libera um bloco previamente alocado

`mymemory_display`: exibe todas as alocações ativas com offset e tamanho

`mymemory_stats`: exibe estatísticas gerais: blocos ativos, memória livre, fragmentos

`mymemory_cleanup`: libera todos os recursos e encerra o pool

1.2 SOLUÇÃO

O pool de memória é alocado uma única vez via `malloc` dentro de `mymemory_init`. A partir desse ponto, a função `mymemory_alloc` nunca chama `malloc` para servir requisições do usuário: ela apenas calcula um offset dentro do pool já existente e retorna um ponteiro para aquela região.

As únicas chamadas adicionais a malloc/free são para gerenciar os nós `allocation_t` da lista encadeada de controle, estrutura interna que rastreia quais regiões do pool estão ocupadas.

2. CODIGO

O núcleo do alocador é a função estática `find_gap`, que percorre a lista encadeada (mantida ordenada por endereço) e identifica as lacunas livres entre blocos alocados:

```
static char *find_gap(mymemory_t *memory, size_t size)
{
    char *prev_end = (char *)memory->pool;
    allocation_t *cur = memory->head;

    for (;;) {
        char *gap_start = prev_end;
        char *gap_end = (cur != NULL) ? (char *)cur->start
                                     : (char *)memory->pool + memory->total_size;
        size_t gap_sz = (size_t)(gap_end - gap_start);

        if (gap_sz >= size) {
            switch (memory->strategy) {
                case STRATEGY_FIRST_FIT: return gap_start;           // primeira lacuna suficiente
                case STRATEGY_BEST_FIT: /* menor lacuna suficiente */ break;
                case STRATEGY_WORST_FIT: /* maior lacuna disponível */ break;
            }
        }

        if (cur == NULL) break;
        prev_end = (char *)cur->start + cur->size;
        cur = cur->next;
    }

    return chosen; // NULL se não houver espaço
}
```

A função varre todas as lacunas (espaço antes do primeiro bloco, entre blocos consecutivos e após o último) e aplica o critério da estratégia configurada. O resultado é passado diretamente para `mymemory_alloc`, que insere um novo nó `allocation_t` na lista e retorna o ponteiro ao chamador.

2.1 TESTES

Temos os 3 arquivos de testes dado pelo professor e tem um que fiz para um teste extra.

test1_first_fit, test1_best_fit, test1_worst_fit: verificam as funcionalidades básicas de cada estratégia — inicialização, alocações simples, liberação e reutilização de lacunas, ponteiros inválidos, alocação maior que o pool, alocação zero e limpeza completa.

test2_stress: estresse com 50 blocos de tamanhos variados (3 a 500 bytes), executado nas três estratégias. Aloca 50 blocos, libera os de índice par (criando 25 lacunas fragmentadas) e realoca 10 novos blocos, verificando que cada estratégia escolhe a lacuna correta.

```
Resultados após alocar 50 blocos e liberar 25 (estado fragmentado):
```

Estratégia	Alocações ativas	Memória alocada	Maior bloco livre	Fragmentos livres
First Fit	25	2285 bytes	4382 bytes	26
Best Fit	25	2285 bytes	4382 bytes	26
Worst Fit	25	2285 bytes	4382 bytes	26

```
Resultados após realocação de 10 blocos (diferença entre estratégias):
```

Estratégia	Fragmentos livres	Maior bloco livre
First Fit	23	4382 bytes
Best Fit	20	4382 bytes
Worst Fit	26	4307 bytes

A diferença no número de fragmentos evidencia o comportamento esperado: Best Fit consolida melhor o espaço ao escolher a menor lacuna suficiente, enquanto Worst Fit distribui as alocações nos maiores buracos, aumentando a fragmentação, mas preservando lacunas maiores para alocações futuras.

3. CONCLUSÃO

A biblioteca implementa corretamente as três estratégias de alocação sobre um pool de memória pré-alocado. O código foi verificado com AddressSanitizer e LeakSanitizer (fsanitize=address,leak) e não apresentou nenhum vazamento de memória ou acesso inválido. Os resultados dos testes confirmam que cada estratégia

se comporta conforme esperado, com diferenças mensuráveis na fragmentação resultante.